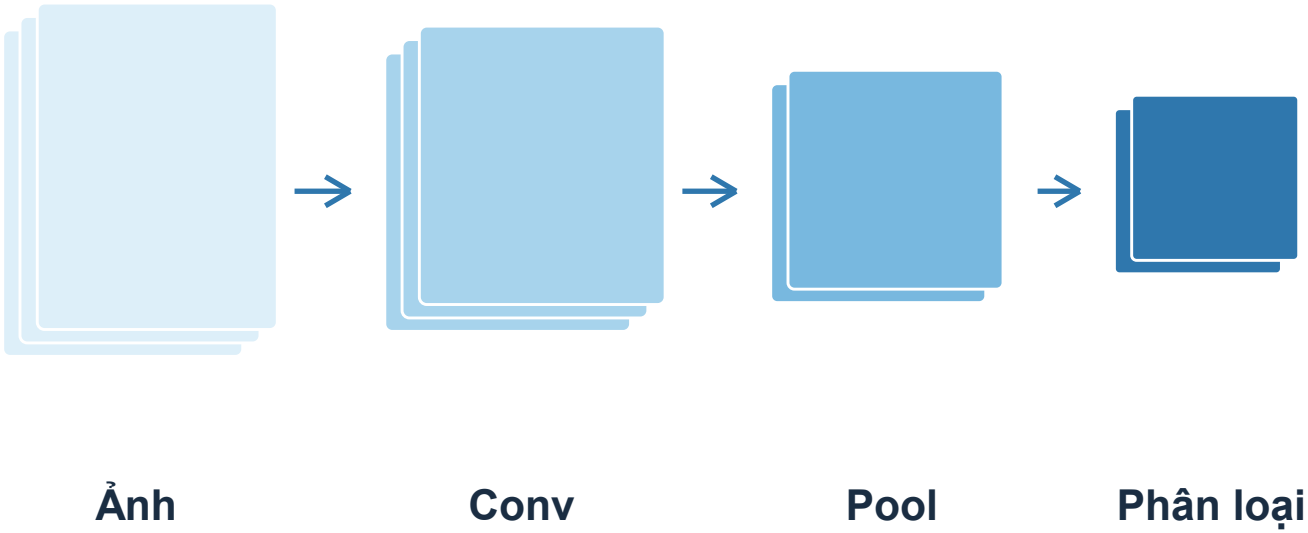
BUỔI 3

CNN: tích chập, pooling và phân loại ảnh

- Hiểu tích chập, kernel, padding, stride, feature map và pooling.
- Phân biệt MLP và CNN trong xử lý ảnh.
- Xây dựng CNN cơ bản bằng PyTorch.
- Thực hành CNN trên FashionMNIST/CIFAR-10 subset.

MINH HỌA • CNN: tích chập, pooling và phân loại ảnh

SƠ ĐỒ / HÌNH MINH HỌA



VÍ DỤ THỰC TẾ

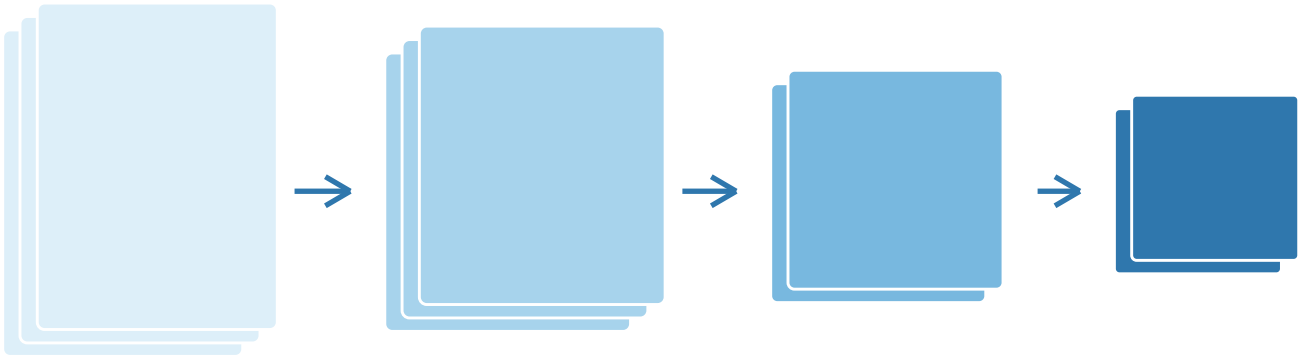
Dùng ảnh CIFAR-10 để học cách phát hiện vật thể từ cạnh, màu và hình dạng thay vì trải phẳng pixel.

Vì sao MLP chưa đủ tốt cho ảnh?

- MLP flatten ảnh nên làm mất quan hệ không gian giữa các pixel lân cận.
- Ảnh có tính cục bộ: cạnh, góc, texture thường nằm trong vùng nhỏ.
- Ảnh có tính lặp: cùng một cạnh có thể xuất hiện ở nhiều vị trí khác nhau.
- CNN đưa inductive bias vào mô hình: local connectivity và weight sharing.
- Kết quả: ít tham số hơn MLP lớn và khai thác cấu trúc ảnh tốt hơn.

MINH HỌA • Vì sao MLP chưa đủ tốt cho ảnh?

SƠ ĐỒ / HÌNH MINH HỌA

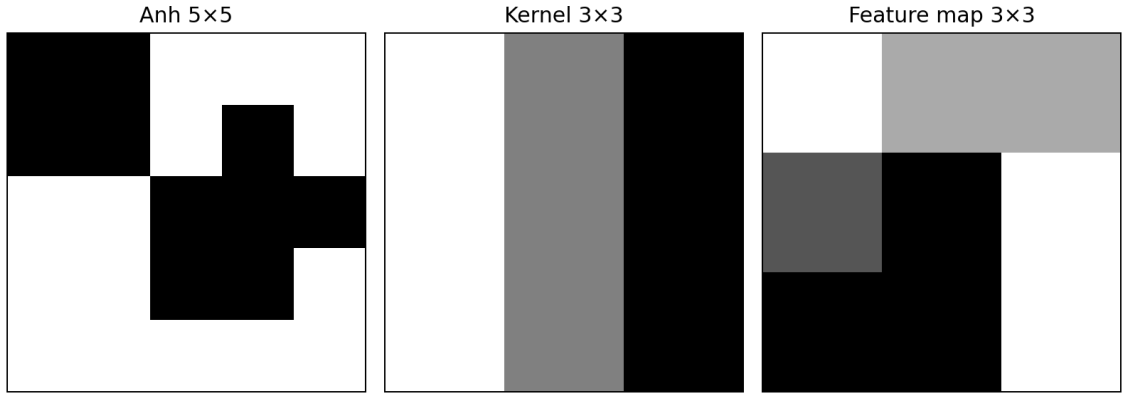


VÍ DỤ THỰC TẾ

Mắt và mũi của chú mèo là cụm pixel gần nhau; CNN khai thác quan hệ này tốt hơn MLP flatten.

Tích chập là gì?

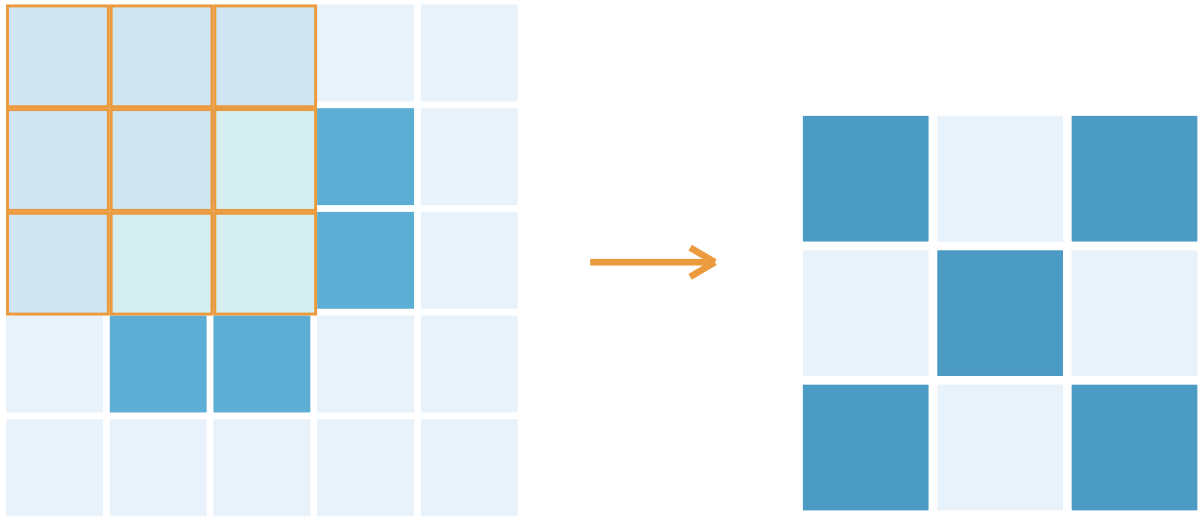
- Kernel/filter là ma trận nhỏ trượt trên ảnh hoặc feature map.
- Tại mỗi vị trí, kernel nhân với vùng ảnh tương ứng rồi cộng lại.
- Kết quả tạo thành feature map biểu diễn mức kích hoạt của đặc trưng.
- Một filter có thể học phát hiện cạnh dọc, cạnh ngang, texture hoặc pattern phức tạp hơn.
- Nhiều filter tạo ra nhiều kênh đặc trưng ở cùng một tầng.



Hình minh họa từ giáo trình

MINH HỌA • Tích chập là gì?

SƠ ĐỒ / HÌNH MINH HỌA



Ảnh/feature map **Feature map đầu ra**

Một kernel dùng lại ở nhiều vị trí trên ảnh

VÍ DỤ THỰC TẾ

Một bộ lọc 3×3 quét ảnh để phát hiện biên đen–trắng, hỗ trợ nhận biết đường nét đồ vật.

Padding và stride

- Padding thêm viền quanh ảnh để kiểm soát kích thước đầu ra.
- Stride là bước trượt của kernel; stride lớn làm giảm kích thước feature map.
- Output size phụ thuộc input size, kernel size, padding và stride.
- Nếu output size sai, các lớp sau sẽ lỗi shape hoặc mismatch linear layer.
- Khi debug CNN, hãy in shape sau từng block.

Giả sử CNN tạo ra 32 feature map kích thước 5×5, sau Flatten sẽ có:
32x5x5=800 đặc trưng
Nếu lớp tiếp theo khai báo nn.Linear(288, 10), chương trình sẽ báo lỗi
vì cần 288 đặc trưng nhưng nhận 800.

Khái niệm	Ý nghĩa
Padding	Thêm viền (thường là số 0) quanh ảnh để giữ hoặc điều chỉnh kích thước đầu ra, đồng thời xử lý thông tin gần mép ảnh.
Stride	Số pixel mà kernel dịch chuyển sau mỗi lần tính toán. Stride = 1 dịch 1 pixel; stride = 2 dịch 2 pixel.
Output size	Kích thước feature map sau tích chập, phụ thuộc kích thước ảnh, kernel, padding và stride.
Shape mismatch	Lỗi kích thước dữ liệu không khớp với đầu vào mà lớp tiếp theo yêu cầu.
Debug CNN	In kích thước tensor sau mỗi lớp để xác định vị trí xảy ra lỗi.

- Padding: Thêm viền để kiểm soát kích thước.
- Stride: Bước trượt của kính lúp (kernel).
- Shape: Kiểm tra kích thước dữ liệu sau mỗi lớp để tránh lỗi khi ghép mạng.

2. Ví dụ: Ảnh 5×5, kernel 3×3

Công thức tính kích thước đầu ra (mỗi chiều):

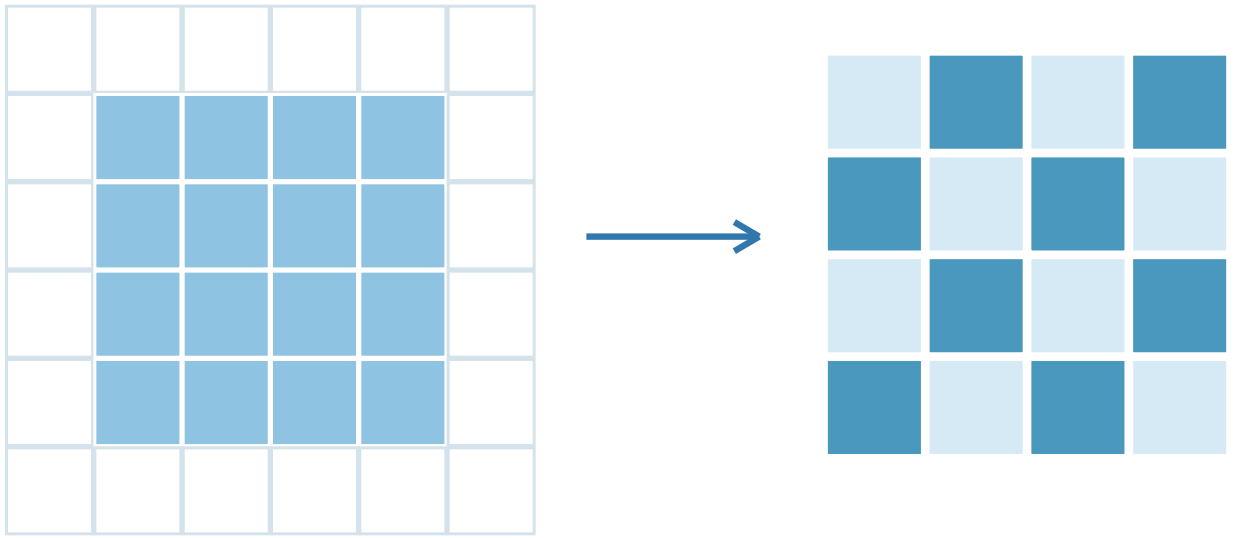
$$O = \left\lfloor \frac{I + 2P - K}{S} \right\rfloor + 1$$

Trong đó: I là kích thước đầu vào, P là padding, K là kernel size và S là stride.

Padding	Stride	Feature map đầu ra
0	1	3×3
1	1	**5×5** (giữ nguyên kích thước)
0	2	2×2
1	2	3×3

MINH HỌA • Padding và stride

SƠ ĐỒ / HÌNH MINH HỌA



Padding = 1

Stride = bước trượt

VÍ DỤ THỰC TẾ

Ảnh 32×32 qua kernel 3×3, padding 1, stride 1 thì vẫn giữ kích thước không gian 32×32.

Pooling và giảm kích thước

Khái niệm	Hiểu là…
Pooling	Thu nhỏ feature map, giữ thông tin quan trọng và giảm khối lượng tính toán.
Max Pooling	Lấy giá trị lớn nhất trong mỗi vùng, giữ đặc trưng nổi bật nhất.
Average Pooling	Lấy giá trị trung bình trong mỗi vùng, giúp làm mượt đặc trưng.
Không có tham số học	Pooling sử dụng phép tính cố định, không có trọng số cần huấn luyện như kernel của CNN.
Mất thông tin	Nếu giảm kích thước quá nhiều, các chi tiết nhỏ có thể bị loại bỏ.

Giả sử một vùng feature map có kích thước 2×2:

Vùng đầu vào 2×2

1	2
3	8

Kết quả pooling

Max Pooling	8
Average Pooling	3,5

- **Max Pooling:** $\max(1, 2, 3, 8) = 8$.
- **Average Pooling:** $(1 + 2 + 3 + 8)/4 = 3,5$.

Với pooling 2×2 và stride = 2, một feature map 4×4 sẽ được thu nhỏ thành 2×2.

Vì Pooling 2×2 với stride = 2 có nghĩa là:

- Pooling 2×2: Mỗi lần lấy một vùng gồm 4 giá trị (2 hàng × 2 cột).
- Stride = 2: Sau mỗi lần tính, vùng pooling dịch chuyển 2 ô, không chồng lấn.

Ví dụ: Feature map 4×4

Đầu vào 4×4

1	3	2	4
5	8	6	2
3	1	9	4
2	6	5	7

Max Pooling → 2×2

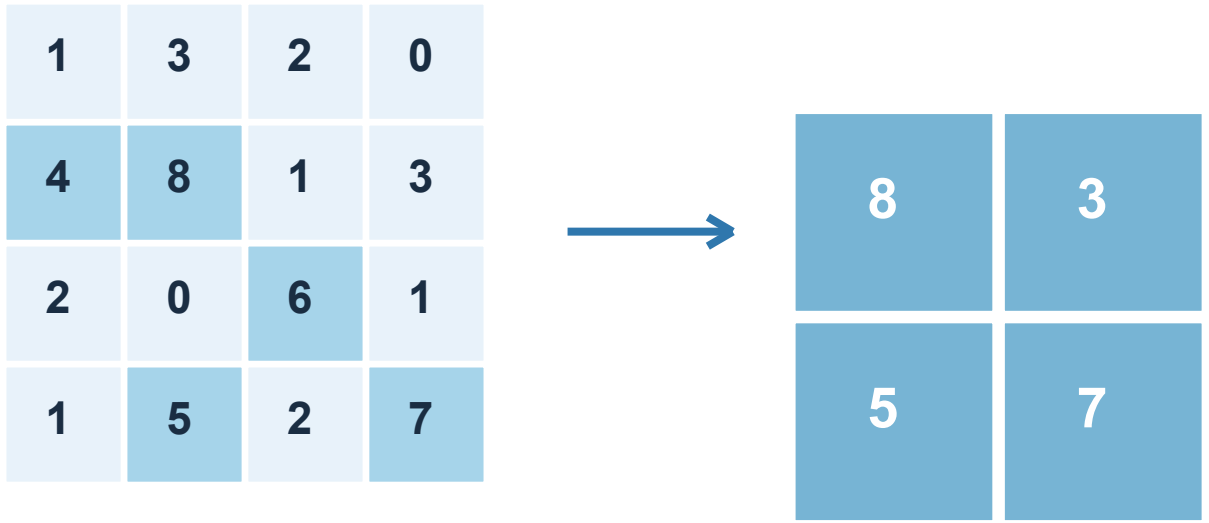
8	6
6	9

Mỗi ô giữ giá trị lớn nhất của vùng 2×2 tương ứng.

Cách tính: 4×4 được chia thành 4 vùng 2×2, mỗi vùng tạo ra 1 giá trị → kết quả gồm 4 giá trị, sắp xếp thành 2×2.

MINH HỌA • Pooling và giảm kích thước

SƠ ĐỒ / HÌNH MINH HỌA



Max pooling 2×2

VÍ DỤ THỰC TẾ

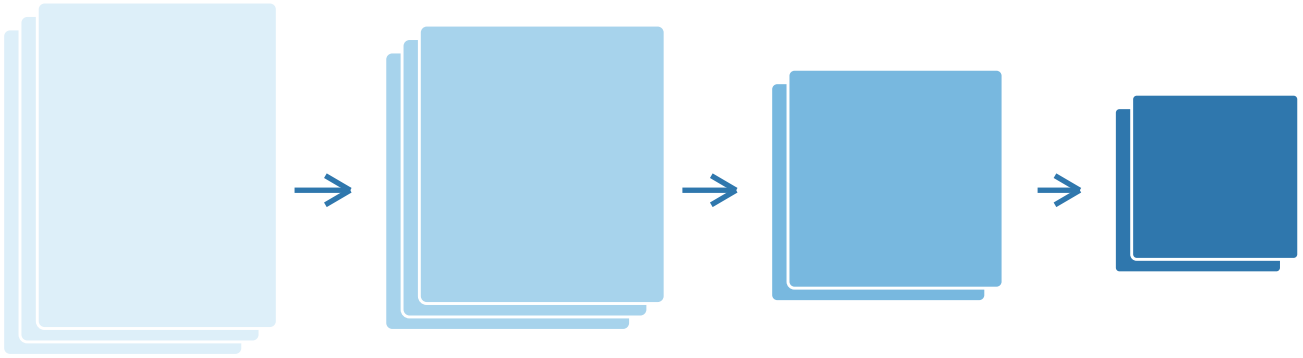
Max pooling 2×2 rút từ 4 điểm ảnh đặc trưng thành 1 giá trị mạnh nhất.

Kiến trúc CNN cơ bản

- Một block phổ biến: Conv2d → ReLU → Pooling.
- Sau nhiều block, feature map được flatten hoặc global average pooling.
- Lớp fully connected cuối cùng chuyển đặc trưng thành logits của các lớp.
- Kích thước channel thường tăng dần: 1/3 → 16 → 32 → 64.
- Cần tính đúng kích thước đầu vào của Linear sau CNN block.

MINH HỌA • Kiến trúc CNN cơ bản

SƠ ĐỒ / HÌNH MINH HỌA

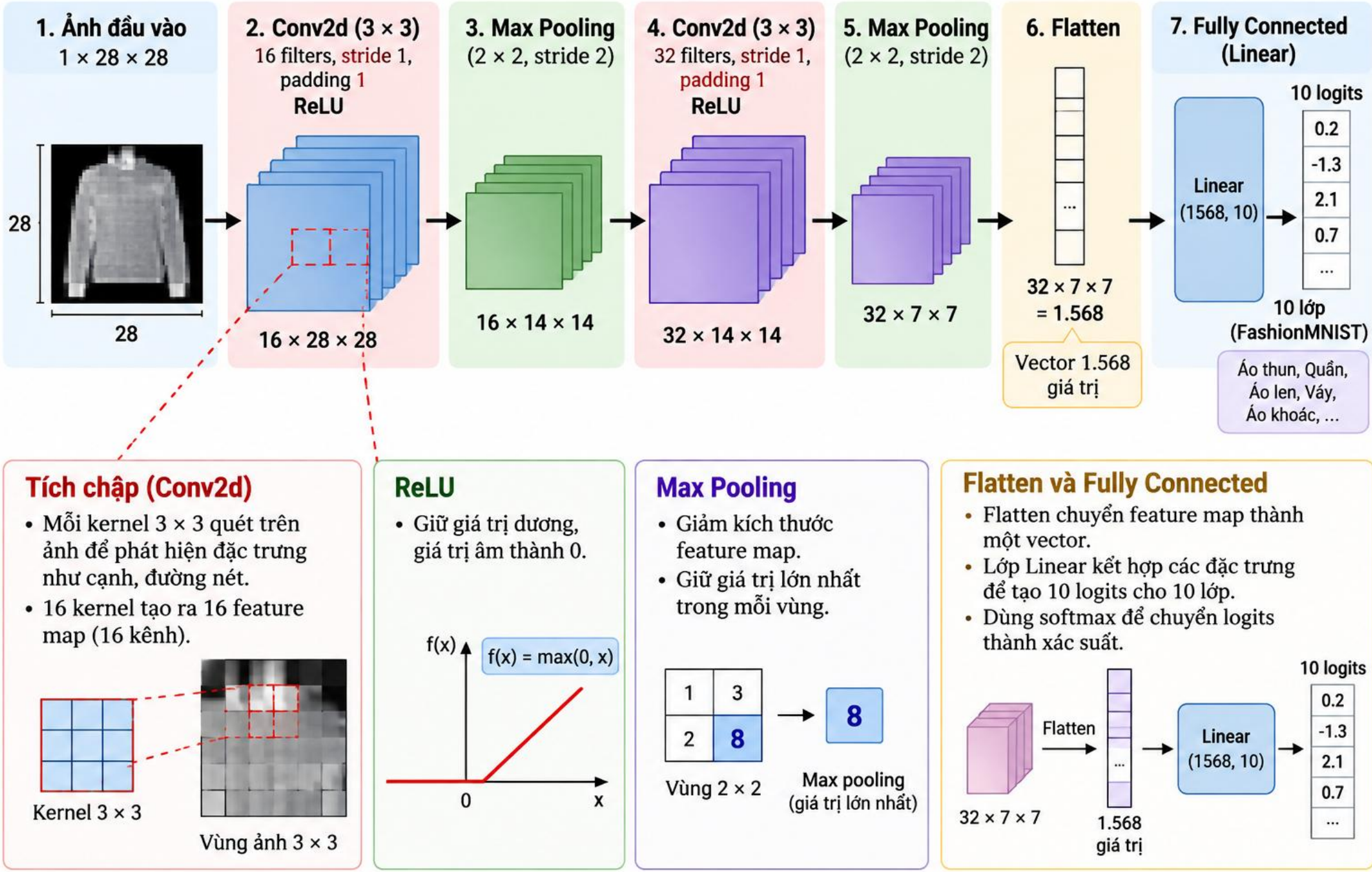


VÍ DỤ THỰC TẾ

Ảnh biển báo → Conv/ReLU/Pool → nhiều feature map → lớp phân loại biển báo.

Feature extraction và classification

- Phần đầu CNN học đặc trưng ảnh: cạnh, texture, hình dạng cục bộ.
- Phần cuối phân loại dựa trên vector đặc trưng đã trích xuất.
- Trong mô hình sâu hơn, đặc trưng tầng thấp thường tổng quát hơn đặc trưng tầng cao.
- Đây là nền tảng để hiểu transfer learning ở buổi 4.
- Câu hỏi: đặc trưng phát hiện cạnh có phụ thuộc nhiều vào dataset không?


1. Ảnh đầu vào
1 × 28 × 28
28
28
2. Conv2d (3 × 3)
16 filters, stride 1, padding 1
ReLU
16 × 28 × 28
3. Max Pooling
(2 × 2, stride 2)
16 × 14 × 14
4. Conv2d (3 × 3)
32 filters, stride 1, padding 1
ReLU
32 × 14 × 14
5. Max Pooling
(2 × 2, stride 2)
32 × 7 × 7
6. Flatten
32 × 7 × 7 = 1.568
Vector 1.568 giá trị
7. Fully Connected (Linear)
Linear (1568, 10)
10 logits
0.2
-1.3
2.1
0.7
...
10 lớp (FashionMNIST)
Áo thun, Quần, Áo len, Váy, Áo khoác, ...
Tích chập (Conv2d)
• Mỗi kernel 3 × 3 quét trên ảnh để phát hiện đặc trưng như cạnh, đường nét.
• 16 kernel tạo ra 16 feature map (16 kênh).
Kernel 3 × 3
Vùng ảnh 3 × 3
ReLU
• Giữ giá trị dương, giá trị âm thành 0.
f(x)
f(x) = max(0, x)
0
x
Max Pooling
• Giảm kích thước feature map.
• Giữ giá trị lớn nhất trong mỗi vùng.
1
3
2
8
8
Vùng 2 × 2
Max pooling (giá trị lớn nhất)
Flatten và Fully Connected
• Flatten chuyển feature map thành một vector.
• Lớp Linear kết hợp các đặc trưng để tạo 10 logits cho 10 lớp.
• Dùng softmax để chuyển logits thành xác suất.
32 × 7 × 7
Flatten
1.568 giá trị
Linear (1568, 10)
10 logits
0.2
-1.3
2.1
0.7
...

Một tensor ảnh trong CNN thường được mô tả theo:

$$\boxed{C \times H \times W}$$

Trong đó:

- C (*Channels*): Số kênh đặc trưng.
- H (*Height*): Chiều cao.
- W (*Width*): Chiều rộng.

Ví dụ, **16×14×14** nghĩa là có **16 feature map**, mỗi feature map có kích thước **14×14**.

Theo dõi ảnh FashionMNIST qua CNN

Ảnh gốc — 1 × 28 × 28

1 kênh ảnh xám, kích thước 28×28 pixel.

Conv2d (16 filters) — 16 × 28 × 28

16 filter tạo 16 feature map. Padding = 1 giữ nguyên chiều cao, chiều rộng.

MaxPool2d (2×2) — 16 × 14 × 14

Pooling làm chiều cao và chiều rộng giảm một nửa; số kênh vẫn là 16.

Conv2d (32 filters) — 32 × 14 × 14

32 filter tạo 32 feature map, kích thước không gian vẫn giữ nguyên.

MaxPool2d (2×2) — 32 × 7 × 7

Mỗi feature map 14×14 giảm còn 7×7.

Flatten — 1.568 giá trị

32 × 7 × 7 = 1.568, chuyển thành vector.

Linear — 10 logits

10 điểm số tương ứng 10 loại quần áo của FashionMNIST.

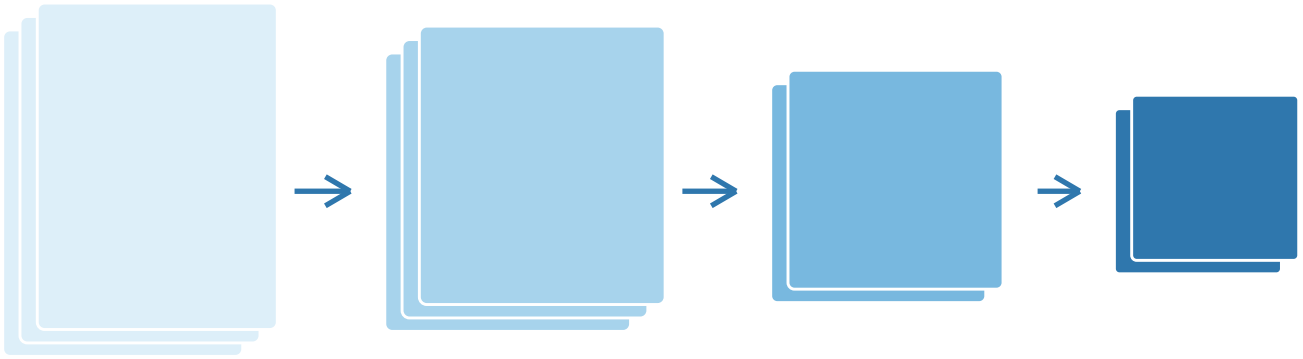
```
model = nn.Sequential(
    nn.Conv2d(1, 16, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),

    nn.Conv2d(16, 32, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),

    nn.Flatten(),
    nn.Linear(32 * 7 * 7, 10)
)
```

MINH HỌA • Feature extraction và classification

SƠ ĐỒ / HÌNH MINH HỌA



VÍ DỤ THỰC TẾ

Tầng đầu phát hiện nét cạnh; tầng sau ghép chúng thành hình bánh xe hoặc khuôn mặt.

Metric cho phân loại ảnh

- Accuracy cho biết tỷ lệ dự đoán đúng tổng thể.
- Confusion matrix giúp phát hiện lớp nào bị nhầm lẫn nhiều.
- Precision/recall hữu ích khi có lớp quan trọng hơn hoặc dữ liệu lệch.
- Quan sát ảnh dự đoán sai giúp phân biệt lỗi mô hình với lỗi nhãn/dữ liệu.
- Báo cáo tốt cần có cả metric số và ví dụ lỗi trực quan.

MINH HỌA • Metric cho phân loại ảnh

SƠ ĐỒ / HÌNH MINH HỌA



Hàng: nhãn đúng | Cột: nhãn dự đoán

VÍ DỤ THỰC TẾ

Bộ phân loại chó–mèo nhầm chó nhỏ thành mèo; xem recall cho lớp chó chứ không chỉ accuracy.

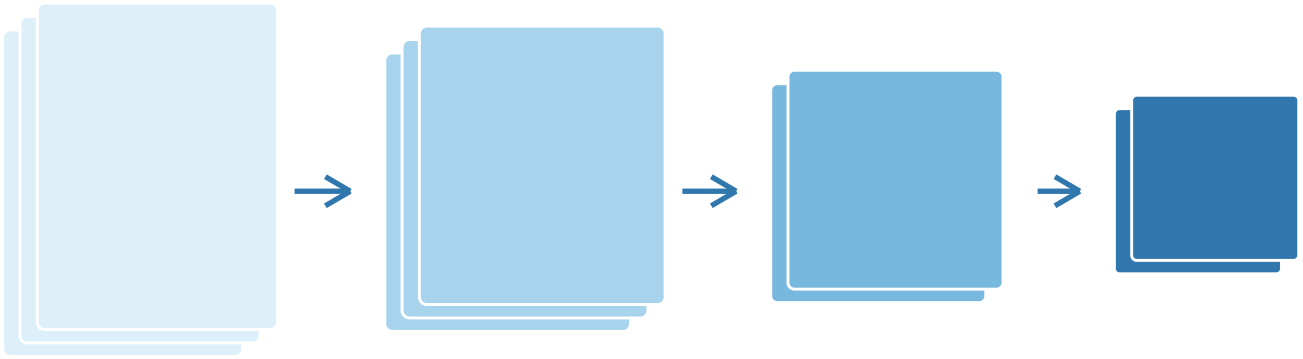
Chỉ số/số liệu minh họa để giải thích trực giác.

Ví dụ: CNN trên FashionMNIST

- Input shape thường là [batch, 1, 28, 28].
- Conv2d cần input 4 chiều: batch, channel, height, width.
- Sau conv/pooling, cần flatten trước khi vào Linear nếu không dùng global pooling.
- CrossEntropyLoss dùng logits kích thước [batch, num_classes].
- Lỗi phổ biến: nhầm channel-first của PyTorch với channel-last của một số thư viện khác.

MINH HỌA • Ví dụ: CNN trên FashionMNIST

SƠ ĐỒ / HÌNH MINH HỌA



VÍ DỤ THỰC TẾ

Ảnh đầu vào [32,1,28,28] là 32 ảnh xám; sau CNN xuất logits [32,10].

Thực hành 1

• File gợi ý: bai03_class_cnn_fashionmnist.py
1. Dùng FashionMNIST (6.000 train, 1.200 validation).
2. CNN hai block Conv2d → ReLU → MaxPool2d(2).
3. In shape sau conv1, pool1, conv2, pool2 và trước Linear (kèm batch dimension).
4. So sánh MLP `784→256→10` với CNN bằng số tham số, validation accuracy và thời gian trung bình/epoch; giữ cùng seed, data split, optimizer, lr, batch và epochs.
5. In `device`, `train loss`, `validation accuracy` và số giây mỗi epoch; sử dụng đúng `model.train()`/`model.eval()`.
6. Nhận xét 3–5 câu về kết quả thực tế và hình dạng tensor.

sản phẩm nộp: Mã Python hoàn thiện, nhật ký shape, bảng MLP–CNN, learning curve và nhận xét ngắn

Thực hành 2

File đề bài:bai03_homework_cnn_variants.py
Dữ liệu: CIFAR-10 hoặc FashionMNIST nếu máy yếu. 6.000 train + 1.200 validation từ official train; test chính thức dành riêng cho đánh giá cuối.
Yêu cầu :
1. Huấn luyện CNN baseline hai block; báo cáo train/validation loss, validation accuracy; lựa chọn mô hình theo validation rồi tính test accuracy và confusion matrix.
2. Thay **kernel size*** (3→5) hoặc **số channel** (16→8 hoặc 32) và phân tích kết quả; giữ nguyên các điều kiện khác.
3. Bổ sung BatchNorm hoặc Dropout (ví dụ 0.3), so sánh với baseline.
4. Trực quan: learning curves, confusion matrix: **hàng nhãn thật – cột nhãn dự đoán**, **5 ảnh đúng + 5 ảnh sai** trên tập test, ghi nhãn thật và nhãn dự đoán.
Sản phẩm nộp: Mã nguồn, bảng cấu hình/kết quả, learning curves, confusion matrix, 5 ảnh đúng và 5 ảnh sai, phân tích 5–10 dòng.